arXiv:1812.10926v1 [cs.DB] 28 Dec 2018

# HUOPM: High Utility Occupancy Pattern Mining

Wensheng Gan, Jerry Chun-Wei Lin*, Philippe Fournier-Viger,
Han-Chieh Chao, *Senior Member, IEEE* and Philip S. Yu, *Fellow, IEEE*

***Abstract*—Mining useful patterns from varied types of databases is an important research topic, which has many real-life applications. Most studies have considered the frequency as sole interestingness measure for identifying high quality patterns. However, each object is different in nature. The relative importance of objects is not equal, in terms of criteria such as the utility, risk, or interest. Besides, another limitation of frequent patterns is that they generally have a low occupancy, i.e., they often represent small sets of items in transactions containing many items, and thus may not be truly representative of these transactions. To extract high quality patterns in real-life applications, this paper extends the occupancy measure to also assess the utility of patterns in transaction databases. We propose an efficient algorithm named High Utility Occupancy Pattern Mining (HUOPM). It considers user preferences in terms of frequency, utility, and occupancy. A novel Frequency-Utility tree (FU-tree) and two compact data structures, called the utility-occupancy list and FU-table, are designed to provide global and partial downward closure properties for pruning the search space. The proposed method can efficiently discover the complete set of high quality patterns without candidate generation. Extensive experiments have been conducted on several datasets to evaluate the effectiveness and efficiency of the proposed algorithm. Results show that the derived patterns are intelligible, reasonable and acceptable, and that HUOPM with its pruning strategies outperforms the state-of-the-art algorithm, in terms of runtime and search space, respectively.**

***Index Terms*—utility mining, interesting pattern, utility theory, utility occupancy.**

## I. Introduction

FREQUENT pattern mining (FPM) and association rule mining (ARM) [1], [2], [3] are some of the important and fundamental data mining techniques to extract meaningful and useful information from massive amounts of data [4], [5]. FPM is the process of discovering frequent sets of items in transaction databases based on a user-specified minimum *support* threshold. In recent decades, the task of frequent pattern mining has been extensively studied by mainly considering the frequency measure for selecting patterns. Other properties and interestingness measures of frequent patterns and association rules have also been studied [3] to fulfill the need of discovering more interesting patterns in databases, e.g., maximal frequent patterns [6], all-confidence [7], coherence [7], and other constraints [8].

In real-life applications, the importance of objects or patterns is often evaluated in terms of implicit factors such as the utility, interestingness, weight, risk or profit [9], [10]. Hence, the knowledge that actually matters to the user may not be found using traditional FPM and ARM algorithms. To measure the utility of patterns, a utility-based mining framework called high utility pattern mining (HUPM) [10] was proposed, which considers the relative importance of items (*item utility*). It has become an emerging research topic in recent years [11], [12], [13], [14]. Chan et al. [15] first introduced the problem of utility-based pattern mining based on business objectives. Yao et al. [16] then defined utility mining as the problem of finding profitable itemsets while considering both the purchase quantities of objects/items in transactions (*internal utilities*) and their unit profits (*external utilities*). In addition, the utility (i.e., importance, interest or risk) of each object/item can be predefined based on users' background knowledge or preferences.

Recently, a study [17] has shown that considering the *occupancy* of patterns is critical for many applications. The occupancy measure is used to ensure that each pattern found represents a large part of transactions where it appears. This allows to find patterns that are more representative, and thus of higher quality. In many applications, occupancy is an interestingness factor to measure a pattern's completeness; and it is an indispensable complement to frequency (or support). However, implicit factors such as the utility, interestingness, risk or profit of objects or patterns are ignored in *occupancy pattern mining* [17], and it ignores the fact that items/objects may appear more than once in transactions. On the other hand, HUPM does not assess the occupancy of patterns. As a result, the discovered patterns may be irrelevant or even misleading if they are not representative of the supporting transactions. For example, if a pattern occurs in many transactions but its actual occupancy is low, it may be inappropriate to use this pattern for recommendation. Hence, it is desirable to find patterns that are representative of the transactions where they occur. In particular, extracting patterns that occupy a large portion of the utility in their supporting transactions is critical to several applications. Recently, an algorithm called OCEAN was proposed to address the problem of high utility occupancy pattern mining by introducing the *utility occupancy* measure [18]. However, it fails to discover the complete set of high utility occupancy patterns (HUOPs) and also encounter several performance problems.

This research was partially supported by the Shenzhen Technical Project under JCYJ20170307151733005 and KQJSCX20170726103424709. Corresponding author: Jerry Chun-Wei Lin, Email: jerrylin@ieee.org

Wensheng Gan is with the School of Computer Science and Technology, Harbin Institute of Technology (Shenzhen), Shenzhen, China, and with the Department of Computer Science, University of Illinois at Chicago, IL, USA. (Email: wsgan001@gmail.com)

Jerry Chun-Wei Lin is with the Department of Computing, Mathematics, and Physics, Western Norway University of Applied Sciences, Bergen, Norway. (Email: jerrylin@ieee.org)

Philippe Fournier-Viger is with the School of Natural Sciences and Humanities, Harbin Institute of Technology (Shenzhen), Shenzhen, China. (Email: philfv@hitsz.edu.cn)

Han-Chieh Chao is with the Department of Electrical Engineering, National Dong Hwa University, Hualien, Taiwan (Email: hcc@ndhu.edu.tw)

Philip S. Yu is with the Department of Computer Science, University of Illinois at Chicago, IL, USA. (Email: psyu@uic.edu)

Therefore, this paper proposes an effective and efficient algorithm named **H**igh **U**tility **O**ccupancy **P**attern **M**ining (**HUOPM**). The proposed algorithm extracts patterns based on users' interests, pattern frequency and utility occupancy, and considers that each item may have a distinct utility. The concept of *utility occupancy* is adopted to evaluate the utility contribution of patterns in their supporting transactions. Thus, the concept of HUOP is quite different from the previous concept of high utility pattern. The previous study [17] shows that the *occupancy* of patterns is very critical to some applications. Compared with *occupancy*, *utility occupancy* is suitable and more effective for pattern analysis in some real-life domains, for example: market basket analysis [1], [2], [14], print-area recommendation for Web pages [17], Web click analysis [11], mobile service provider [18], and biomedical applications [19].

Consider the travel route recommendation for tourist to visit, eat, and spend time/money, HUOPM can successfully exploit the frequency and utility contribution ratio (w.r.t. utility occupancy) of a specific travel route. For another example, a mobile service provider can obtain some real-world data, and each record contains the traffic information that a customer spends on various mobile Apps. In general, each App has its unit utility (i.e., price, popular). In this case, the concept of *utility occupancy* and the HUOPM model can be applied to discover the high qualified patterns (both frequent and high utility occupancy). After identifying the set of mobile Apps that users are interested in (e.g., download frequently, spend most of their money), this information can also be utilized to improve the service. The major contributions of this paper are summarized as follows.

- A novel and effective HUOPM algorithm is proposed to address the novel research problem of mining high utility occupancy patterns with the utility occupancy measure. To the best of our knowledge, no prior algorithms address this problem successfully and effectively.
- Two compact data structures called **U**tility-**O**ccupancy list (UO-list) and **F**requency-**U**tility table (FU-table), are designed to store the required information about a database, for mining HUOP. When necessary and sufficient conditions are met, information about a pattern can be directly obtained from the built UO-lists of its prefix patterns, and thus HUOPM can avoid repeatedly scanning the database.
- Moreover, the concept of remaining utility occupancy is utilized to calculate an upper bound to reduce the search space. Based on the developed pruning strategies, the HUOPM algorithm can directly discover HUOPs from the designed frequency-utility tree using UO-lists by only scanning a database twice.
- Extensive experiments have been conducted on both real-world and synthetic datasets to evaluate how effective and efficient the proposed HUOPM algorithm is. Results show that HUOPM can reveal the desired useful patterns, having a high utility occupancy and predefined frequency, and that the proposed pruning strategies are effective at leading to a more compact search space.

The rest of this paper is organized as follows. Related work is reviewed in Section II. In Section III, some key preliminaries are introduced and the addressed problem is defined. The proposed HUOPM algorithm and several pruning strategies are described in Section IV. Then, to evaluate the effectiveness and efficiency of the HUOPM algorithm, results of extensive experiments comparing its performance with the state-of-the-art algorithm are provided in Section V. Finally, conclusions and future work are drawn in Section VI.

## II. Related Work

### A. Support-based Pattern Mining

In numerous domains, web mining and data mining technologies provide powerful ways of discovering interesting and useful information in massive amounts of data. In recent decades, the task of support-based frequent pattern mining has been widely studied. Most algorithms have been designed to extract patterns using the frequency (support) measure as selection criterion [1], [2], [3]. Among them, Apriori [1] and FP-growth [2] are two of the most well-known frequent pattern mining algorithms. Other properties and interestingness measures of frequent patterns and association rules have also been studied to fulfill the need for discovering more interesting patterns [3], [7], [8]

In the past, significant progress have been made concerning data mining with support and confidence [3]. Many algorithms have been proposed to discover high quality patterns from the binary transaction databases [3], [7], [8], [20]. However, these approaches do not consider that objects/items may occur more than once in a transaction (e.g., may have non binary purchase quantities), and that objects/items are often not equally important to the user (e.g., may not have the same unit profit). Up to now, associations of boolean attributes only considered in traditional ARM, which only reflects the frequency of the presence or absence of an item in the database. Thus, the problem of quantitative association rule mining (QARM) has been studied [21], [22], [23]. In QARM, the attributes of item can be quantitative (e.g., age, income, purchase quantity) instead of the boolean value (0 or 1). However, QARM still does not reflect the other important factors of items, such as interest, price, risk or profit. In some real-life applications, the frequency (support) of a pattern may be an inappropriate measure to determine the importance of this pattern.

### B. Utility-based Pattern Mining

To address the limitations of support-based pattern mining and to extract high profitable patterns, the high-utility pattern mining (HUPM) task has been studied [11], [12], [13], [14]. HUPM considers both the occur quantities and unit profits of objects/items rather than just considering their occurrence frequencies. A pattern is concerned as a high-utility pattern (HUP) if its utility is no less than the predefined minimum utility threshold. Chan et al. [15] presented a framework to mine the top-$k$ closed utility patterns. Yao et al. [16] then defined utility mining as the problem of discovering profitable patterns while considering both the purchase quantity of objects/items in transactions (*internal utility*) and their unit profits (*external utility*). They then introduced the mathematical properties of utility constraint to respectively reduce the search space and

expected utility upper bounds [24]. The concept of HUPM is different from the support-based weighted frequent pattern mining (WFPM) [9], [25], [26], because WFPM does not consider the quantity information of each object/item, and the range of weight is [0,1]. By taking both quantities and profits of items into consideration, HUPM can reveal more valuable patterns than frequent ones. Liu et al. then presented a transaction-weighted utilization (TWU) model [27] to discover HUPs by adopting a transaction-weighted downward closure (TWDC) property. In recent years, developing algorithms for mining high utility patterns is an active research topic. Some recent algorithms for HUPM are IHUP [11], UP-growth [13], UP-growth+ [14], HUI-Miner [12], d2HUP [28], FHM [29], HUP-Miner [30], and EFIM [31].

Different from the efficiency issue of HUPM, there are also a number of studies that focus on some interesting effectiveness issues of HUPM. For example, mining high utility patterns from different types of data (i.e., uncertain data [32], [33], temporal data [34], [35], transaction data with negative unit profits [36], dynamic data [37], [38], [39], [40], and stream data [41]), HUPM with various discount strategies [42], non-redundant correlated HUPs [43], HUPM using multiple minimum utility thresholds [32], a condensed set of HUPs [44], discriminative HUPs [45], top-$k$ issue of HUPM [44], and HUPM from big data [46]. All these HUPM algorithms discover high utility patterns based on the basic definitions of utility mining model by Yao et al. [16], [24]. In recent years, there has been a growing interest in utility-oriented mining models for discovering different types of high utility patterns (e.g., itemsets, rules, sequences, and episodes) and profitable information. The comprehensive survey of this research field can be referred to [10], [40].

### C. High Quality Pattern Mining

Although HUPM methods can evaluate the utility of patterns, they do not assess the occupancy of patterns in their supporting transactions in terms of utility. In other words, HUPM methods do not consider how important the utility of patterns are compared to the utility of the transactions where they appear. For some real-life applications, such as pattern-based recommendation, it is desirable that interesting patterns should occupy a large portion of the transactions where they appear [17]. The above approaches fail to meet this requirement. To address this issue, a novel measure called *utility occupancy* was proposed in the OCEAN algorithm [18]. As a relative measure, utility occupancy ensures that a certain set of items is important to individual users.

However, the OCEAN algorithm suffers from two important drawbacks. First, the mining results derived by OCEAN are incomplete. The reason is that the exact utility information is incorrectly kept in the utility-list [12] structure using an inconsistent sorting order. As a result, OCEAN applies pruning strategies with incorrect information. Second, OCEAN is not efficient as it does not utilize the support property and utility occupancy property well to prune the search space. As a result, it performs poorly when the related parameters are set low. Hence, developing an effective and efficient algorithm to address these limitations is important. In this study, the problem of effectively mining the complete set of all high utility occupancy patterns from a database is discussed.

## III. Preliminaries and Problem Statement

Let $I = \{i_1, i_2, \ldots, i_m\}$ be a finite set of $m$ distinct items in a transactional quantitative database $D = \{T_1, T_2, \ldots, T_n\}$, where each quantitative transaction $T_q \in D$ is a subset of $I$, and has a unique identifier *tid*. The total utility of all items in a transaction is named transaction utility and denoted as *tu*. An itemset $X$ with $k$ distinct items $\{i_1, i_2, \ldots, i_k\}$ is called a $k$-itemset. A database consisting of 10 transactions and 5 distinct items is shown in TABLE I, which will be used as a running example.

TABLE I
An example quantitative database.

| *tid* | Transaction (item, quantity) | *tu* |
|---|---|---|
| $T_1$ | *a*:2, *c*:4, *d*:7 | $65 |
| $T_2$ | *b*:2, *c*:3 | $37 |
| $T_3$ | *a*:3, *b*:2, *c*:1, *d*:2 | $38 |
| $T_4$ | *b*:4, *d*:3 | $11 |
| $T_5$ | *a*:1, *b*:3, *c*:2, *d*:5, *e*:1 | $49 |
| $T_6$ | *c*:2, *e*:4 | $58 |
| $T_7$ | *c*:2, *d*:1 | $23 |
| $T_8$ | *a*:3, *b*:1, *d*:2, *e*:4 | $61 |
| $T_9$ | *a*:2, *c*:4, *d*:1 | $59 |
| $T_{10}$ | *c*:3, *e*:1 | $42 |

*Definition 1:* The number of transactions containing an itemset is said to be its occurrence frequency or *support count* [1], [2]. The support count of an itemset $X$, denoted as $sup(X)$, is the number of *supporting transactions* containing $X$. Let the set of transactions supporting an itemset $X$ denote as $\Gamma_X$. A transaction $T_q$ is said to support an itemset $X$ if $X \subseteq T_q$. Thus, $sup(X) = |\Gamma_X|$. Let the user-specified minimum support threshold denote as $\alpha$, $X$ is called a frequent pattern (*FP*) in a database $D$ if $sup(X) \geq \alpha \times |D|$.

*Definition 2:* Each item $i_m$ in a database $D$ has a unit profit, denoted as $pr(i_m)$, which represents its relative importance to the user. Item unit profits are indicated in a user-specified profit table, denoted as $ptable = \{pr(i_1), pr(i_2), \ldots, pr(i_m)\}$. The utility of an item $i_j$ in a transaction $T_q$ is defined as $u(i_j, T_q) = q(i_j, T_q) \times pr(i_j)$, in which $q(i_j, T_q)$ is the occur quantity of $i_j$ in $T_q$. The utility of an itemset/pattern $X$ in a transaction $T_q$ is defined as $u(X, T_q) = \sum_{i_j \in X \wedge X \subseteq T_q} u(i_j, T_q)$. Thus, the total utility of $X$ in a database $D$ is $u(X) = \sum_{X \subseteq T_q \wedge T_q \in D} u(X, T_q)$.

*Example 1:* In TABLE I, assume that the unit profit of items $(a)$ to $(e)$ are defined as $\{pr(a)$:\$7, $pr(b)$:\$2, $pr(c)$:\$11, $pr(d)$:\$1, $pr(e)$:\$9$\}$, respectively. Consider the itemsets $(a)$ and $(ab)$, their utilities in $T_3$ are $u(a, T_3) = 3 \times \$7 = \$21$, and $u(ab, T_3) = 3 \times \$7 + 2 \times \$2 = \$21 + \$4 = \$25$, respectively. Thus, their utilities in the database are calculated as $u(a) = u(a, T_1) + u(a, T_3) + u(a, T_5) + u(a, T_8) + u(a, T_9) = \$14 + \$21 + \$7 + \$21 + \$14 = \$77$, and $u(ab) = u(ab, T_3) + u(ab, T_5) + u(ab, T_8) = \$21 + \$13 + \$23 = \$57$.

*Definition 3:* The transaction utility (*tu*) of a transaction $T_q$ is $tu(T_q) = \sum_{i_j \in T_q} u(i_j, T_q)$, where $i_j$ is the $j$-th item in $T_q$.

*Example 2:* $tu(T_1) = u(a, T_1) + u(c, T_1) + u(d, T_1)$ = \$14 + \$44 + \$7 = \$65. The transaction utilities of transactions $T_1$ to $T_{10}$ are respectively calculated as *tu*($T_1$) = \$65, *tu*($T_2$) = \$37, *tu*($T_3$) = \$38, *tu*($T_4$) = \$11, *tu*($T_5$) = \$49, *tu*($T_6$) = \$58, *tu*($T_7$) = \$23, *tu*($T_8$) = \$61, *tu*($T_9$) = \$59, and *tu*($T_{10}$) = \$42, as shown in TABLE I.

In the study [17], a new interestingness measure called *occupancy* was proposed to discover frequent patterns having a strong occupancy. A new concept named *utility occupancy* [18] is first introduced below. It is important to notice that the *utility occupancy* concept is different from the original *occupancy* concept presented in the DOFIA algorithm [17].

*Definition 4:* The utility occupancy of an itemset $X$ in a supporting transaction $T_q$ is denoted as $uo(X, T_q)$, and defined as the ratio of the utility of $X$ in that transaction divided by the total utility of that transaction:

$$uo(X, T_q) = \frac{u(X, T_q)}{tu(T_q)}. \quad (1)$$

*Example 3:* Since $tu(T_1)$ = \$65 and $tu(T_3)$ = \$38, the utility occupancy of $(ac)$ in $T_1$ is calculated as $uo(ac, T_1)$ = \$58/\$65 $\approx$ 0.8923, and the utility occupancy of $(ac)$ in $T_3$ is calculated as $uo(ac, T_3)$ = \$32/\$38 $\approx$ 0.8421.

*Definition 5:* The utility occupancy of an itemset $X$ in a database $D$ is denoted as $uo(X)$, and defined as:

$$uo(X) = \frac{\sum_{X \subseteq T_q \wedge T_q \in D} uo(X, T_q)}{|\Gamma_X|}, \quad (2)$$

where $\Gamma_X$ is the set of *supporting transactions* of $X$ in $D$ (thus $|\Gamma_X|$ is equal to the support of $X$ in $D$).

Hence, utility occupancy can be used to evaluate how patterns contribute to the total utility of transactions where they appears in. In prior work [17], the occupancy of a pattern in a transaction was defined as the ratio between the number of items in this pattern and the number of items in the transaction. The utility occupancy of a pattern generalizes this definition to consider that items may have distinct utility values, and is defined as the harmonic average of the occupancy utility values in all supporting transactions. Therefore, the two concepts, occupancy and utility occupancy, are different in nature.

*Definition 6:* Given a minimum support threshold $\alpha$ ($0 < \alpha \leq 1$) and a minimum utility occupancy threshold $\beta$ ($0 < \beta \leq 1$), an itemset $X$ in a database $D$ is said to be a high utility occupancy pattern with high frequency and strong utility occupancy, denoted as ***HUOP***, if it satisfies the following two conditions: $sup(X) \geq \alpha \times |D|$ and $uo(X) \geq \beta$.

*Example 4:* The utility occupancies of $(a)$ and $(ab)$ in TABLE I are calculated as: $uo(a)$ = ($uo(a, T_1)$ + $uo(a, T_3)$ + $uo(a, T_5)$ + $uo(a, T_8)$ + $uo(a, T_9)$)/5 = (0.2154 + 0.5526 + 0.1429 + 0.3443 + 0.2373)/5 $\approx$ 0.2985, $uo(ab)$ = ($uo(ab, T_3)$ + $uo(ab, T_5)$ + $uo(ab, T_8)$)/3 = (0.6579 + 0.2653 + 0.3371)/3 = 0.4201. When $\alpha$ and $\beta$ are set to 30% and 0.30, the complete set of HUOPs in the running example database is: $(c)$, $(e)$, $(ab)$, $(ac)$, $(ad)$, $(bc)$, $(bd)$, $(cd)$, $(ce)$, $(abd)$, $(acd)$, as shown in TABLE II. Clearly, the utility occupancy measure does not respect the *downward closure* property of Apriori in FPM.

Note that the minimum support count must be set such that $\alpha \times |D| > 1$. Otherwise, each transaction (or an itemset) will have a utility occupancy value of 1. In the example of TABLE I, if $\alpha$ is set to 0.1, then ($\alpha \times |D|$) = 1, and all ten transactions are HUOPs. For example, $T_1$ (i.e., the itemset $(acd)$) has a support count as 1 and a utility occupancy as 1.0, thus satisfying the two conditions of HUOP. Based on the above definitions, the problem statement of high utility occupancy pattern mining (HUOPM) is formulated as follows:

TABLE II
THE DERIVED HUOPS.

| Pattern | *sup* | *uo* | Pattern | *sup* | *uo* |
|---|---|---|---|---|---|
| $(c)$ | 8 | 0.6468 | $(bd)$ | 4 | 0.3620 |
| $(e)$ | 4 | 0.4022 | $(cd)$ | 5 | 0.6881 |
| $(ab)$ | 3 | 0.4334 | $(ce)$ | 3 | 0.8776 |
| $(ac)$ | 4 | 0.8273 | $(abd)$ | 3 | 0.4959 |
| $(ad)$ | 5 | 0.3609 | $(acd)$ | 4 | 0.8972 |
| $(bc)$ | 3 | 0.6554 | | | |

**Problem Statement.** Given a transaction database $D$, a profit-table indicating the distinct profit of each item, a minimum support threshold $\alpha$, and a minimum utility occupancy threshold $\beta$. The problem of mining high utility occupancy patterns (which are both frequent and dominant in terms of high utility occupancy) is to discover the complete set of patterns that not only have a frequency no less than $\alpha \times |D|$, but also have a utility occupancy no less than $\beta$.

## IV. PROPOSED ALGORITHM FOR MINING HUOPS

In the section, we propose two compact data structures called utility-occupancy list (UO-list) and frequency-utility table (FU-table) to maintain the utility occupancy information about a database. Then we utilize both the support and utility occupancy measures to prune the search space for mining the more interesting and useful high utility occupancy patterns.

### A. Search Space for Mining HUOPs

According to previous studies, the search space for the pattern mining problem can be represented as a lattice structure [47] or as a Set-enumeration tree [48]. Based on the final derived HUOPs of the given example, it can be observed that the well-known *downward closure* property of Apriori does not hold for HUOPs. For example, 1-items $(a)$, $(b)$ and $(d)$ are not HUOPs, but their supersets $\{(ab)$, $(ac)$, $(ad)$, $(bc)$, $(bd)$, $(cd)$, $(abd)$, $(acd)\}$ are HUOPs, as shown in TABLE II. If no anti-monotone property is applied in HUOPM, a huge number of candidates will need to be generated to obtain the actual HUOPs. It is thus a critical issue to design more suitable data structures and powerful pruning strategies to efficiently reduce the search space and to filter the number of unpromising patterns for mining HUOPs.

*Definition 7:* **(*Total order* $\prec$ *on items*)** Without loss of generality, assume that items in every transaction are sorted according to the lexicographic order. Furthermore, assume that the total order $\prec$ on items in the designed HUOPM algorithm adopts the support ascending order of items.

Note that the total order $\prec$ used in the proposed HUOPM algorithm can be the support-based ascending or descending order, the TWU-based ascending or descending order, the

lexicographic order, or any other total order on items. The processing order of patterns may, however, affect the mining efficiency of HUOPM, but does not affect the completeness and correctness of this algorithm. We assume that the support ascending order is adopted in the HUOPM algorithm. Notice that the efficiency when using different orders in HUOPM will be conducted and evaluated in the experiments. For example, the support counts of items in the running example are $\{sup(a)$:5, $sup(b)$:5, $sup(c)$:8, $sup(d)$:7, $sup(e)$:4$\}$. Thus, the support ascending order for five items is $\{sup(e) < sup(a) \leq sup(b) < sup(d) < sup(c)\}$, and the total order $\prec$ on items is $e \prec a \prec b \prec d \prec c$.

*Definition 8:* ***(Frequency-utility tree, FU-tree)*** A frequency-utility tree (FU-tree) is a variant of sorted Set-enumeration tree using the total order $\prec$ on items. It contains the frequency and utility information.

*Definition 9:* **(*Extension nodes in the FU-tree*)** In the designed FU-tree, all child nodes of any tree node are called its extension nodes.

Clearly, the designed FU-tree is a utility-based prefix-tree, and the complete search space can be traversed using a depth-first search (DFS) or breadth-first search (BFS), where each child node in the FU-tree is generated by extending its prefix (parent) node. Consider the running example with the adopted total order $\prec$ on items, the extension nodes of node $(ab)$ are the itemsets $(abd)$, $(abc)$ and $(abdc)$. Note that all the supersets of node $(ab)$ are $(eab)$, $(abd)$, $(abc)$, $(eabd)$, $(eabc)$, $(abde)$ and $(eabde)$. Hence, the extension nodes of a tree node in a FU-tree are a subset of the supersets of that node.

### B. UO-List and FU-Table

Two compact data structures, called utility-occupancy list (UO-list for short) and frequency-utility table (FU-table for short), are designed to keep essential information about patterns in the database. Note that the UO-list is different from the utility-list structure used in HUI-Miner [12], FHM [29] and OCEAN [18]. The occupancy information is not stored in utility-list, while UO-list keeps this information to quickly calculate the utility occupancy of a pattern using a join operation. Besides, a new concept called *remaining utility occupancy* is introduced and applied to obtain an utility occupancy upper-bound, which will be presented in the next subsection.

*Definition 10:* The remaining utility occupancy of an itemset $X$ in a transaction $T_q$ is denoted as $ruo(X, T_q)$, and defined as the sum of the utility occupancy values of all items appearing after $X$ in $T_q$ according to the total order $\prec$ , that is:

$$ruo(X, T_q) = \frac{\sum_{i_j \notin X \wedge X \subseteq T_q \wedge X \prec i_j} u(i_j, T_q)}{tu(T_q)}. \quad (3)$$

For example, consider $T_5$ and itemsets $(a)$ and $(ad)$ in TABLE I. Since the total order $\prec$ on items is $e \prec a \prec b \prec d \prec c$, we have that $ruo(a, T_5)$ = ($u(b, T_5)$ + $u(d, T_5)$ + $u(c, T_5)$)/$tu(T_5)$ = (\$6 + \$5 + \$22)/\$49 $\approx$ 0.6735, and $ruo(ad, T_5)$ = $(u(c, T_5))/tu(T_5)$ = \$22/\$49 $\approx$ 0.4490.

*Definition 11:* **(*Utility-occupancy list, UO-list*).** Let $\prec$ be the total order on items from $I$. The *utility-occupancy list* of an itemset $X$ in a database $D$ is a set of tuples corresponding to transactions where $X$ appears. A tuple contains three elements $<$***tid***, ***uo***, ***ruo***$>$ for each transaction $T_q$ containing $X$. In each tuple, the ***tid*** element is the transaction identifier of $T_q$; the ***uo*** element is the utility occupancy of $X$ in $T_q$, w.r.t. $uo(X, T_q)$; and the ***ruo*** element is defined as the remaining utility occupancy of $X$ in $T_q$, w.r.t. $ruo(X, T_q)$.

*Example 5:* Consider the running example and the defined total order $\prec$ ($\{e \prec a \prec b \prec d \prec c\}$). The constructed UO-lists of five 1-itemsets (itemsets of length 1) are shown in Fig. 1. Note that the UO-lists of all 1-itemsets are constructed after the HUOPM algorithm performs a single database scan.

**(e)**

| *tid* | *uo* | *ruo* |
|---|---|---|
| 5 | 0.1837 | 0.8163 |
| 6 | 0.6207 | 0.3793 |
| 8 | 0.5902 | 0.4098 |
| 10 | 0.2143 | 0.7857 |

**(a)**

| *tid* | *uo* | *ruo* |
|---|---|---|
| 1 | 0.2154 | 0.7846 |
| 3 | 0.5526 | 0.4474 |
| 5 | 0.1429 | 0.6735 |
| 8 | 0.3443 | 0.0656 |
| 9 | 0.2373 | 0.7627 |

. . . . . .

**(c)**

| *tid* | *uo* | *ruo* |
|---|---|---|
| 1 | 0.6769 | 0 |
| 2 | 0.8919 | 0 |
| 3 | 0.2895 | 0 |
| 5 | 0.4490 | 0 |
| 6 | 0.3793 | 0 |
| 7 | 0.9565 | 0 |
| 9 | 0.7458 | 0 |
| 10 | 0.7857 | 0 |

Fig. 1. Constructed UO-lists of the five items.

To discover HUOPs, the sum of the support and utility occupancy of a special pattern $X$ in a database $D$ can be efficiently calculated by adding the utility occupancies of all elements in the UO-list of $X$ (denoted as *X.UOL*). Thus, the following information can be obtained from *X.UOL*: (1) the name of $X$; (2) the set of transactions where $X$ appears (its support); (3) the sum of the utility occupancy of $X$ in $D$; and (4) the total remaining utility occupancy of $X$ in $D$. To explain how this useful information can be obtained from the constructed UO-lists, the following definitions are introduced by utilizing the UO-list structure. We further design a data structure called frequency-utility table (FU-table) by utilizing some useful properties of UO-list. The FU-table of an expected pattern is built after the construction of UO-list of this pattern, and it stores the following information.

*Definition 12:* **(*FU-table*).** A frequency-utility table (FU-table) of an itemset $X$ contains four informations: the name of the itemset $X$ (***name***), the support of $X$ (***sup(X)***), the sum of the utility occupancies of $X$ in database $D$ (***uo(X)***), and the sum of the remaining utility occupancies of $X$ in $D$ (***ruo(X)***). Here, $ruo(X)$ can be calculated as:

$$ruo(X) = \frac{\sum_{X \subseteq T_q \wedge T_q \in D} ruo(X, T_q)}{|\Gamma_X|}. \quad (4)$$

The construction process of a FU-table is shown below. Consider an item $(e)$ in TABLE I, which appears in $T_5$, $T_6$, $T_8$, and $T_{10}$. The built UO-list of $(e)$ is shown in Fig. 2(a). The FU-table of item $(e)$ is constructed efficiently by using the support count, utility occupancy and remaining utility occupancy. They are calculated during the construction of the UO-list of $(e)$, such that $\{sup(e)$ = 4, $uo(e)$ = (0.1837 + 0.6207 + 0.5902 + 0.2143)/4 = 0.4022, and $ruo(e)$ = (0.8163 + 0.3793 + 0.4098 + 0.7857)/4 = 0.5978$\}$, and the results of

its FU-table are $\{sup(e)$ = 4, $uo(e)$ = 0.4022, and $ruo(e)$ = 0.5978$\}$, as shown in Fig. 2(b). Thus, the final built FU-tables of all items (1-itemsets) are shown in Fig. 3.

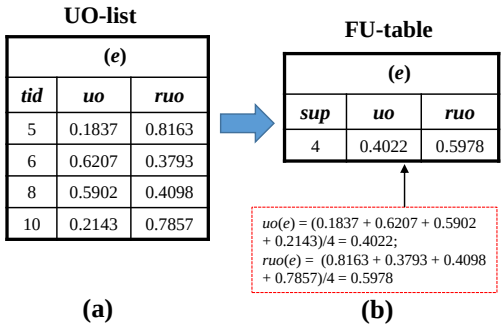



Fig. 2. The UO-list and FU-table of item (e).

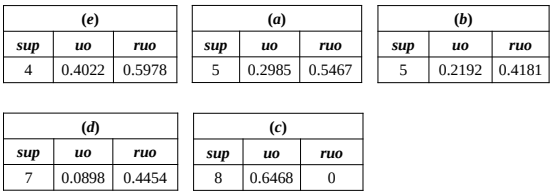

(e)
sup uo ruo
4 0.4022 0.5978
(a)
sup uo ruo
5 0.2985 0.5467
(b)
sup uo ruo
5 0.2192 0.4181
(d)
sup uo ruo
7 0.0898 0.4454
(c)
sup uo ruo
8 0.6468 0

Fig. 3. Constructed FU-tables of all 1-itemsets.

According to the two conditions $sup(X) \geq \alpha \times |D|$ and $uo(X) \geq \beta$ of HUOP, we only need to initially construct the UO-lists and FU-tables of frequent 1-itemsets ($FP^1$). Then, for any $k$-itemset ($k \geq 2$), the UO-list of this itemset can be calculated directly using the UO-lists of some of its subsets, without scanning the database. The construction procedure of the UO-list and FU-table of a $k$-itemset is shown in Algorithm 1. Given an itemset $X$, and two itemsets $X_a$ and $X_b$ which are extensions of $X$ obtained by adding two distinct items $a$ and $b$ to $X$, respectively. The construction procedure takes as input the UO-lists of $X$, $X_a$ and $X_b$, and outputs the UO-list and FU-table of the itemset $X_{ab}$. Note that $X_{ab}.UOL$ denote the UO-list of $X_{ab}$, and $X_{ab}.FUT$ denote the FU-table of $X_{ab}$.

In Algorithm 1, Lines 17 to 20 show the developed pruning strategy named *remaining support strategy*, which will be described later. It is important to notice that the construction of the UO-list and FU-table of a $k$-itemset ($k \geq 3$, Lines 5 to 8) is different than that of a 2-itemset ($k = 2$, Lines 10 to 12). Details of the difference are described in the following content. The $uo$ of $X_{ab}$ is the sum of the $uo$ associated with $tid$ in the UO-lists of $X_a$ and $X_b$. Suppose $a$ is before $b$, and then the $ruo$ of $X_{ab}$ is assigned as the $ruo$ associated with $tid$ in the UO-list of $X_b$. However, when calculating the $uo$ and $ruo$ for a $(k)$-itemset ($k \geq 2$), the part of $uo$ is different, thus it needs to subtract the $uo$ of the common part between $X_a$ and $X_b$. Generally, to calculate the $uo$ of $\{i_1 \dots i_{(k-2)} i_{(k-1)} i_k\}$ in $tid$, the following formula holds: $uo(\{i_1 \dots i_{(k-2)} i_{(k-1)} i_k\}, tid)$ = $uo(\{i_1 \dots i_{(k-2)} i_{(k-1)}\}, tid)$ + $uo(\{i_1 \dots i_{(k-2)} i_k\}, tid)$ - $uo(\{i_1 \dots i_{(k-2)}\}, tid)$. For example, in transaction $T_5$, $uo(ac, T_5)$ = $uo(a, T_5)$ + $uo(c, T_5)$ = 0.1429 + 0.4490 = 0.5919, $uo(ae, T_5)$ = $uo(a, T_5)$ + $uo(e, T_5)$ = 0.1429 + 0.1837 = 0.3266, and $uo(ace, T_5)$ = $uo(ac, T_5)$ + $uo(ae, T_5)$ - $uo(a, T_5)$ = 0.5919 + 0.3266 - 0.1429 = 0.7756. We can not miscalculate the $uo$ of $\{ace\}$ in $T_5$ such as the sum of the $uo$ of $\{ac\}$ and $\{ae\}$ in $T_3$, since they contain the $uo$ of $\{a\}$ in $T_5$ twofold.

The procedure can be easily implemented since a set of the UO-lists of the $(k$-1)-itemsets ($k \geq 2$) has been built before constructing the UO-list of a $(k)$-itemset ($k \geq 2$) w.r.t. the extension node.

**Algorithm 1** Construction($X$, $X_a$, $X_b$)

```
1:  set X_ab.UOL ← ∅, X_ab.FUT ← ∅;
2:  for each tuple E_a ∈ X_a.UOL do
3:    if ∃E_a ∈ X_b.UOL ∧ E_a.tid == E_b.tid then
4:      if X.UOL ≠ ∅ then
5:        search for E ∈ X.UOL, E.tid = E_a.tid;
6:        E_ab ← < E_a.tid, E_a.uo + E_b.uo - E.uo,
          E_b.ruo >;
7:        X_ab.FUT.uo += E_a.uo + E_b.uo − E.uo;
8:        X_ab.FUT.ruo += E_b.ruo;
9:      else
10:       E_ab ←< E_a.tid, E_a.uo + E_b.uo, E_b.ruo >;
11:       X_ab.FUT.uo += E_a.uo + E_b.uo;
12:       X_ab.FUT.ruo += E_b.ruo;
13:     end if
14:     X_ab.UOL ← X_ab.UOL ∪ E_ab;
15:     X_ab.FUT.sup ++;
16:   else
17:     X_a.FUT.sup - -;
18:     if X_a.FUT.sup < α × |D| then
19:       return null;
20:     end if
21:   end if
22: end for
23: return X_ab
```

### C. Upper Bound on Utility Occupancy

Using the proposed UO-list and FU-table structures, the actual utility occupancy value of a pattern can be calculated exactly. But a crucial question is: is it necessary to construct the UO-lists and FU-tables of all patterns in a FU-tree? Since this would be very expensive in terms of runtime and memory usage, some pruning conditions should be developed to decide whether a subtree must be spanned. In a Set-enumeration tree [48], the complete search space of $I$ (where $m$ is the number of items in $I$) contains $2^m$ patterns (by systematically enumerating all subsets of $I$ with the total order $\prec$, that is all possible patterns). As mentioned before, the *downward closure* property does not hold for the utility occupancy. Thus, it is a crucial challenge to design powerful technologies to prune the search space and filter unpromising patterns early, especially to handle large-scale databases.

Without actually generating all possible high utility occupancy patterns, can we derive an upper bound $\hat{\phi}(X)$ on the utility occupancy of patterns in a subtree rooted at node representing an itemset $X$? Unfortunately, it is not an easy

task since the utility occupancy and the supporting transactions of patterns are unknown until these patterns are processed. Hence, inspired by the OCEAN algorithm [18], we try to develop an upper bound on the utility occupancy by utilizing the UO-list structure. If $\hat{\phi}(X)$ is less than the minimum utility occupancy threshold, we can safely discard this subtree from further consideration.

*Definition 13:* Let there be an itemset $X$ and a transaction (or an itemset) $T$ such that $X \subseteq T$, and that the set of all items in $T$ that are not in $X$ is denoted as $T \setminus X$. In addition, the set of all items appearing after $X$ in $T$ according to the $\prec$ order is denoted as $T/X$. Thus, $T/X \subseteq T \setminus X$.

*Lemma 1:* Let there be a subtree rooted at $X$, $\Gamma_X$ be the supporting transactions of $X$. Then, for any possible high utility occupancy itemset $W$ in the subtree, we have:

$$uo(W) \leq \frac{\sum_{W \subseteq T_q \wedge T_q \in D}(uo(X,T_q) + ruo(X,T_q))}{|\Gamma_W|}. \quad (5)$$

*Proof:* Since $W$ is a $k$-extension of $X$, $\Gamma_W \subseteq \Gamma_X$, we have $(W - X) = (W/X)$. Thus, in each transaction $T_q$, $uo(W/X, T_q) \leq uo(T_q/X, T_q)$. The following relationships can be obtained. For each transaction $T_q$ in $D$, since

$$\begin{aligned} uo(W) &= \frac{\sum_{W \subseteq T_q \wedge T_q \in D} uo(X,T_q)}{|\Gamma_W|} \\ &= \frac{\sum_{W \subseteq T_q \wedge T_q \in D}(uo(X,T_q) + uo(W-X,T_q))}{|\Gamma_W|} \\ &\leq \frac{\sum_{W \subseteq T_q \wedge T_q \in D}(uo(X,T_q) + uo(T_q/X,T_q))}{|\Gamma_W|} \\ \Longrightarrow uo(W) &\leq \frac{\sum_{W \subseteq T_q \wedge T_q \in D}(uo(X,T_q) + uo(T_q/X,T_q))}{|\Gamma_W|}. \\ \Longrightarrow uo(W) &\leq \frac{\sum_{W \subseteq T_q \wedge T_q \in D}(uo(X,T_q) + ruo(X,T_q))}{|\Gamma_W|}. \end{aligned}$$

■

The supporting transactions of a processed node (itemset) $X$ is denoted as $\Gamma_X$, but the supporting transactions of any possible high utility occupancy pattern $W$ extending $X$ w.r.t. $\Gamma_W$, is unknown although $\Gamma_W \subseteq \Gamma_X$. The set $\Gamma_W$ is unknown until $W$ is processed. Assume that we now explore the node $(ea)$. After constructing the UO-list of $(ea)$, we can obtain its support count and utility occupancy. Unfortunately, we do not know the related support count $\Gamma_W$ of any of its extension nodes $W$. Because of Inequality (5), it is difficult to obtain an upper bound for $W$ without knowing $\Gamma_W$. The concept of HUOP indicates that every HUOP should be supported by $\alpha \times |D|$ transactions in the database. By utilizing this property, we further develop the following theorems to obtain an upper bound for a subtree rooted at a processed node in the FU-tree.

*Lemma 2:* Let there be a minimum support threshold $\alpha$, a subtree rooted at $X$, and $\Gamma_X$ be its supporting transactions. For any pattern $W$ in the subtree, an upper bound on the utility occupancy of $W$ is:

$$\hat{\phi}(W) = \frac{\sum_{top\alpha \times |D|, T_q \in \Gamma_X}(uo(X,T_q) + ruo(X,T_q))}{|\alpha \times |D||}. \quad (6)$$

$$\hat{\phi}(W) \geq uo(W). \quad (7)$$

*Proof:* Note that $\Gamma_W \subseteq \Gamma_X$ and $\Gamma_W$ is unknown, we calculate $(uo(X,T_q) + ruo(X,T_q))$ for all transactions in $\Gamma_X$ and put into a vector set (denoted as $V_{occu}$), then sort $V_{occu}$ in descending order (denoted as $V^{\downarrow}_{occu}$). Since the average of top $k$ ($0 < k \leq |\Gamma_W|$) values of vector $V^{\downarrow}_{occu}$ is an upper bound of the average of total $|\Gamma_W|$ values of $\frac{\sum_{W \subseteq T_q \wedge T_q \in D}(uo(X,T_q) + ruo(X,T_q))}{|\Gamma_W|}$. Since a high utility occupancy pattern should be supported by at least $\alpha \times |D|$ transactions, we have $\alpha \times |D| \leq k \leq |\Gamma_W| \leq |\Gamma_X|$. We have,

$$\begin{aligned} uo(W) &\leq \frac{\sum_{W \subseteq T_q \wedge T_q \in D}(uo(X,T_q) + ruo(X,T_q))}{|\Gamma_W|} \\ \Longrightarrow uo(W) &\leq \frac{\sum_{topk, T_q \in \Gamma_X}\{uo(X,T_q) + ruo(X,T_q)\}^{\downarrow}}{|\Gamma_W|} \\ \Longrightarrow uo(W) &\leq \frac{\sum_{top\alpha \times |D|, T_q \in \Gamma_X}\{uo(X,T_q) + ruo(X,T_q)\}^{\downarrow}}{|\alpha \times |D||} \\ \Longrightarrow uo(W) &\leq \hat{\phi}(W). \end{aligned}$$

Thus, given a minimum support threshold $\alpha$, we can directly calculate an upper bond $\hat{\phi}(W)$ on utility occupancy of a subtree which rooted at a processed node $X$. ■

### D. Proposed Pruning Strategies

In this section we will show how to efficiently prune the search space using an upper bound on the utility occupancy and the support count. Two properties named *global downward closure* property and *partial downward closure* property can be obtained below.

*Lemma 3:* The complete search space of the addressed HUOPM problem can be represented by a FU-tree where items are sorted according to the support ascending order on items.

*Proof.* According to the studies in [48], the complete search space for mining HUOPs can be presented as a Set-enumeration tree.

*Theorem 1:* **(*Global downward closure property in the FU-tree*)** In the designed FU-tree, if a tree node is a FP, its parent node is also a FP. Let $X^k$ be a $k$-itemset (node) and its parent node be denoted as $X^{k-1}$, which is a ($k$-1)-itemset. The relationship $sup(X^k) \leq sup(X^{k-1})$ holds.

*Proof:* According to the well-known Apriori property [1], the relationship $sup(X^k) \leq sup(X^{k-1})$ exists. Thus, in the FU-tree, the *global downward closure* property holds. ■

*Theorem 2:* **(*Partial downward closure property in the FU-tree*)** In the designed FU-tree, let $X^k$ be a $k$-itemset (node) and its parent node be denoted as $X^{k-1}$, a ($k$-1)-itemset. The relationship indicating that the upper bound on utility occupancy of any node in a subtree is no greater than that of its parent node always holds, that is $\hat{\phi}(X^k) \leq \hat{\phi}(X^{k-1})$.

*Proof:* Since the UO-list of $X^k$ is constructed by joining the UO-list of $X^{k-1}$ with the one of a sibling of $X^{k-1}$, we have $\Gamma_{X^k} \subseteq \Gamma_{X^{k-1}}$. The average of the top $k$ ($0 < k \leq |\Gamma_{X^k}|$ values in the vector $V^{\downarrow}_{occu}$ of $X^k$ is no greater than the average of top $k$ ($0 < k \leq |\Gamma_{X^{k-1}}|$ values of vector $V^{\downarrow}_{occu}$

of $X^{k-1}$. According to Lemma 2, the relationship between upper bounds on the utility occupancy of $X^k$ and $X^{k-1}$ is: $\hat{\phi}(X^k) \leq \hat{\phi}(X^{k-1})$. As a result, if $\hat{\phi}(X^k) \geq \beta$, then $\hat{\phi}(X^{k-1}) \geq \hat{\phi}(X^k) \geq \beta$. Conversely, if $\hat{\phi}(X^{k-1}) < \beta$, then $\hat{\phi}(X^k) \leq \hat{\phi}(X^{k-1}) < \beta$. In other words, the obtained upper bound on the utility occupancy of patterns satisfies the *partial downward closure* property. Thus, in the developed FU-tree, the *partial downward closure* property holds. ■

It is noteworthy that "*partial*" here indicates that the upper bound on the utility occupancy is conditional anti-monotone but not general anti-monotone. In other words, the *partial downward closure* property holds for a tree node and its descendants, but does not hold for supersets which are not in that subtree. To further enhance the pruning effect of the proposed HUOPM algorithm, the derived upper bound can be utilized with this anti-monotone property. We next present four novel pruning strategies, which were not used in the state-of-the-art OCEAN algorithm [18].

*Strategy 1:* In the designed FU-tree and by considering the defined total order $\prec$, if a tree node $X$ has a support count less than $(\alpha \times |D|)$, then any nodes containing $X$ (i.e., all supersets of $X$) can be directly pruned and do not need to be explored.

*Example 6:* In the running example, assume that the parameters are set to $\alpha$ = 30% and $\beta$ = 0.3. Using the UO-lists of $(e)$ and $(a)$, we can construct the UO-list of $(ea)$. Since $sup(ea) = 2 < (\alpha \times 10 = 30\% \times 10 = 3)$, all extension nodes of $(ea)$ are not HUOPs and can be directly pruned.

*Strategy 2:* In the designed FU-tree, considering the defined total order $\prec$, if the upper bound on the utility occupancy of a tree node $X$ is less than $\beta$, then any nodes in the subtree rooted at $X$ w.r.t. all extensions of $X$ can be directly pruned and do not need to be explored.

*Example 7:* After constructing the UO-list of $(ab)$, we can obtain its utility occupancy (*uo*) and remaining utility occupancy (*ruo*) in each supporting transaction. According to inequality (5), the top 3 values in the sorted vector $V^{\downarrow}_{occu}$ of $(ab)$ are considered. Since the UO-list of $(ab)$ is {($T_3$, 0.6579, 0.3421), ($T_5$, 0.2653, 0.5510), ($T_8$, 0.3771, 0.0328)}, $V^{\downarrow}_{occu}$ of $(db)$ = {($T_3$, 1.0000), ($T_5$, 0.8163), ($T_8$, 0.4099)}. Thus, the utility occupancy upper bound of $(ab)$ is (1.0000 + 0.5380 + 0.4099)/3 $\approx$ 0.7421 $>$ 0.3, and $sup(ab)$ = 3. Thus, its extension nodes $(abd)$, $(abc)$ and $(abdc)$ may be HUOPs, and hence should be explored.

*Strategy 3:* During the construction of the UO-list of a tree node $X_{ab}$ by using $X_a$ and $X_b$, if the remaining support of $X_a$ for constructing $X_{ab}$ is less than $\alpha \times |D|$, the support of $X_{ab}$ will also be less than $\alpha \times |D|$, $X_{ab}$ is not a FP, and also not a HUOP. Then the construction procedure returns *null*, as shown in Algorithm 1 Lines 18 to 20.

*Strategy 4:* After calling the construction procedure to build the UO-list of a tree node $X$, if *X.UOL* is empty or $sup(X) \leq \alpha \times |D|$, $X$ is not a HUOP, and none of its subtree nodes is a HUOP. Then, *X.UOL* is not added to the set of extension UO-lists of $X$.

*Example 8:* In the running example, the node $(ea)$ has a support count of 2. Its subtree nodes can be pruned by Strategy 4. In this case, Strategy 4 has the same effect as Strategy 1.

### *E. Proposed HUOPM Algorithm*

Based on two compact data structures (UO-list and FU-table), an upper bound on utility occupancy and the above pruning strategies, the proposed HUOPM algorithm can be designed below. Algorithm 2 shows the pseudo-code of the designed HUOPM algorithm. It takes four parameters as input, including a transactional database $D$, a predefined profit table *ptable*, a user-specified minimum support threshold $\alpha$ ($0 < \alpha \leq 1$), and a minimum utility occupancy threshold $\beta$ ($0 < \beta \leq 1$). HUOPM first scans the database to calculate the $sup(i)$ of each item $i \in I$ and the $tu$ value of each transaction (Line 1), then finds the set $I^*$ w.r.t. $FP^1$ (Line 2, Strategy 1), and sorts it according to the total order $\prec$ (Line 3). After that, the algorithm scans $D$ again to construct the UO-list and FU-table of each item $i \in I^*$ (Line 4). At last, HUOPM recursively applies the designed ***HUOP-Search*** function for mining HUOPs without generating candidates and without scanning the database repeatedly (Line 5).

**Algorithm 2** HUOPM ($D$, *ptable*, $\alpha$, $\beta$)

1: scan $D$ to calculate the $sup(i)$ of each item $i \in I$ and the $tu$ value of each transaction;
2: find $I^* \leftarrow \{i \in I | sup(i) \geq \alpha \times |D|\}$, w.r.t. $FP^1$;
3: sort $I^*$ in the designed total order $\prec$;
4: using the total order $\prec$, scan $D$ once to build the UO-list and FU-table for each 1-item $i \in I^*$;
5: **call *HUOP-Search***$(\phi, I^*, \alpha, \beta)$.
6: **return** *HUOPs*

**Algorithm 3** HUOP-Search ($X$, *extenOfX*, $\alpha$, $\beta$)

1: **for** each itemset $X_a \in$ *extenOfX* **do**
2: obtain $sup(X_a)$ and $uo(X_a)$ from the built $X_a.FUT$;
3: **if** $sup(X_a) \geq \alpha \times |D|$ **then**
4: **if** $uo(X_a) \geq \beta$ **then**
5: $HUOPs \leftarrow HUOPs \cup X_a$;
6: **end if**
7: $\hat{\phi}(X_a) \leftarrow$ ***UpperBound***$(X_a.UOL, \alpha)$;
8: **if** $\hat{\phi}(X_a) \geq \beta$ **then**
9: *extenOfX*$_a \leftarrow \emptyset$;
10: **for** each $X_b \in$ *extenOfX* that $X_a \prec X_b$ **do**
11: $X_{ab} \leftarrow X_a \cup X_b$;
12: call ***Construct***$(X, X_a, X_b)$;
13: **if** $X_{ab}.UOL \neq \emptyset \wedge sup(X_{ab}) \geq \alpha \times |D|$ **then**
14: *extenOfX*$_a \leftarrow$ *extenOfX*$_a \cup X_{ab}.UOL$;
15: **end if**
16: **end for**
17: **call *HUOP-Search***$(\boldsymbol{X_a}, \textit{extenOfX}_{\boldsymbol{a}}, \boldsymbol{\alpha}, \boldsymbol{\beta})$;
18: **end if**
19: **end if**
20: **end for**
21: **return** *HUOPs*

During the recursive exploration of the search space (i.e., FU-tree) with the total order $\prec$, the partial anti-monotonicity of HUOPs is used to effectively avoid generating unpromising patterns that cannot be HUOPs and their child nodes. Based on the proposed pruning strategies, the upper bound can be used

to prune patterns with low support or low utility occupancy early, without constructing their UO-lists. It can effectively reduce both the computational cost of join operations and the search space in the FU-tree. In fact, the actual search space is more compact than a complete FU-tree.

Details of the ***HUOP-Search*** procedure is shown in Algorithm 3. It takes as input an itemset $X$, a set of UO-lists of all 1-extensions of $X$, $\alpha$ and $\beta$. A loop is first performed over each 1-extension of itemset $X$, denoted as $X_a$ (Lines 1 to 20). Two conditions are checked (Lines 3 to 4, Strategy 1) to determine if $X_a$ is a HUOP. If the itemset is a HUOP, then it is put into the set of HUOPs. The algorithm then determines whether extensions of $X_a$ should be explored (Lines 7 to 8, Strategy 2). It uses the function ***UpperBound*** (c.f. Algorithm 4) to calculate the upper bound of utility occupancy for the subtree rooted at pattern $X_a$. If $\hat{\phi}(X_a) \geq \beta$, then any extensions of $X_a$ may be a HUOP. In that case, the depth-first search is performed (Lines 8 to 18). The construction procedure ***Construct(X, $X_a$, $X_b$)*** is then executed for each 1-extension node $X_{ab}$, to construct their UO-lists and FU-tables (Lines 11 to 12). Note that each constructed itemset $X_{ab}$ is an 1-extension of the itemset $X_a$. If $X_{ab}.UOL \neq \emptyset$ and $sup(X_{ab}) \geq \alpha \times |D|$, such itemset is added to a set *extenOf*$X_a$ for storing all 1-extensions of $X_a$ (Lines 13 to 15, Strategy 4). HUOPM recursively applies the designed ***HUOP-Search*** function for determining the desired HUOPs (Line 17).

**Algorithm 4** UpperBound ($X_q.UOL$, $\alpha$)

1: $sumTopK \leftarrow 0, \hat{\phi}(X_a) \leftarrow 0, V_{occu} \leftarrow \emptyset$;
2: calculate $(uo(X, T_q) + ruo(X, T_q))$ of each tuple from the built $X_a.UOL$ and put them into the set of $V_{occu}$;
3: sort $V_{occu}$ by descending order as $V^{\downarrow}_{occu}$;
4: **for** $k \leftarrow 1$ to $\alpha \times |D|$ in $V^{\downarrow}_{occu}$ **do**
5: $\quad sumTopK \leftarrow sumTopK + V^{\downarrow}_{occu}[k]$;
6: **end for**
7: $\hat{\phi}(X_a) = \dfrac{sumTopK}{\alpha \times |D|}$.
8: **return** $\hat{\phi}(X_a)$

Algorithm 4 provides the pseudo-code of ***UpperBound*** procedure, for computing the upper bound of utility occupancy. The complexity of Algorithm 4 is $O(n \times log(n) \times V^{\downarrow}_{occu}[k])$ (the top $k$ value in vector is equal to $\alpha \times |D|$). We first use $O(n \times log(n))$ time for sorting $V_{occu}$ with quick sort algorithm in Line 3, then use $O(n \times log(n) \times V^{\downarrow}_{occu}[k])$ time for the loop operation from Lines 4 to 7 to calculate the top $k$ values in $V^{\downarrow}_{occu}[k])$ (Lines 4 to 5). Finally, it returns the average value of the summation as the upper bound on utility occupancy of the subtree rooted at pattern $X_a$ (Lines 4 to 5).

## V. Experiments

In this section, extensive experiments are conducted on both real-world and synthetic datasets to evaluate the effectiveness and efficiency of the proposed HUOPM algorithm. The task of the addressed problem is to discover high utility occupancy patterns (HUOPs) with frequency and utility occupancy. Note that only two prior studies, DOFIA [17] and OCEAN [18] are closely related to our work. Major differences are that DOFIA considers both the frequency and occupancy for mining high qualified patterns, but the occupancy is based on the number of items in patterns or transactions rather than the concept of utility [17]. Thus, in the following, the state-of-the-art OCEAN algorithm [18] is implemented to generate high utility occupancy patterns. HUOPs* are derived by OCEAN, and HUOPs are generated by the proposed HUOPM algorithm.

We first evaluate the mining results by comparing HUOPs* and HUOPs. Then the runtime and the number of visited nodes in the search tree of the HUOPM algorithm are compared. At last, the effect of the processing order of items is also compared and evaluated.

### A. Experimental Setup and Datasets

All algorithms in the experiments are implemented using the Java language and executed on a PC with an Intel Core i5-3460 3.2 GHz processor and 4 GB of memory, running on the 32 bit Microsoft Windows 7 platform. Four real-world datasets [49] (BMSPOS2, retail, chess, and mushroom) and two synthetic dataset [50] (T10I4D100K and T40I10D100K) are used in the experiments. T10I4D100K and T40I10D100K are generated using the IBM Quest Synthetic Data Generator [50]. These datasets have varied characteristics and represents the main types of data typically encountered in real-life scenarios (e.g., dense, sparse and long transactions). The characteristics of these datasets are described below in details.

- *BMSPOS2* dataset contains several years worth of point-of-sale data from a large electronics retailer. It has total 515,597 transactions with 1,657 distinct items, an average transaction length of 6.53 items.
- *retail* is a sparse dataset which contains 88,162 transactions with 16,470 distinct items and an average transaction length of 10.30 items.
- *chess* dataset contains 3,196 transactions with 75 distinct items and an average transaction length of 36 items. It is a dense dataset.
- *mushroom* is a dense dataset, it has 8,124 transactions with 120 distinct items, and an average transaction length of 23 items.
- *T10I4D100K* is a synthetic dataset that contains 870 distinct items, 100,000 transactions, and has an average length of 10.1 items.
- *T40I10D100K* is a synthetic dataset that contains 942 distinct items, 100,000 transactions, and has an average length of 39.6 items.

For the addressed utility-based HUOPM problem, note that the quantity and unit profit of each item in a dataset is randomly generated by using a simulation method proposed in previous studies [12], [13], [14]. In the following experiments, the proposed HUOPM algorithm with different designed pruning strategies is respectively denoted as HUOPM$_{P12}$ (with pruning strategies 1 and 2), HUOPM$_{P13}$ (with pruning strategies 1 and 3), HUOPM$_{P123}$ (with pruning strategies 1, 2 and 3) and HUOPM$_{P1234}$ (with all pruning strategies). The four versions are compared to evaluate the efficiency of HUOPM and the effect of the designed pruning strategies.

### B. Pattern Analysis

In the study presenting OCEAN, it was shown that the diversity of patterns with high utility occupancy is more favorable than traditional high utility patterns and frequent patterns. The study used a small real-life dataset *MobileApp* to illustrate that the derived patterns are more useful for the task of mobile App recommendation or promotion [18]. To analyze the usefulness of the proposed HUOPM framework, the derived patterns (HUOPs* and HUOPs) are evaluated below. When setting $\alpha$:20% and $\beta$:0.50 on the running example, the top-10 patterns derived by HUOPM, OCEAN and FP-growth [2] are shown in TABLE III, respectively. Notice that the results in left, middle and right of TABLE III are sorted by the descending order of *utility occupancy* (*uo*) and support (*sup*), respectively. It is clearly that three real HUOPs of top-10 results ($(abde)$, $(abe)$, and $(cd)$, as shown in TABLE III (left)) are missed in the results of OCEAN (as shown in TABLE III (middle)). Besides, the minimum *utility occupancy* in top-10 HUOPs* is 0.5604, while the minimum *utility occupancy* value in top-10 HUOPs is 0.6554. Some interesting desired patterns with high *uo* values cannot be discovered by OCEAN. Therefore, the results derived by the OCEAN algorithm are incomplete and OCEAN also encounters some performance problems (it will be discussed in the next subsection). Finally, it can be clearly seen that the support-based frequent patterns (as shown in TABLE III (right)) are quite different from the results derived by the utility-occupancy-based methods.

TABLE III
DERIVED PATTERNS FROM RUNNING EXAMPLE

| (**left**) Top-10 HUOPs | | | (**middle**) Top-10 HUOPs* | | | (**right**) Top-10 FPs | | |
|---|---|---|---|---|---|---|---|---|
| **Pattern** | ***sup*** | ***uo*↓** | **Pattern** | ***sup*** | ***uo*↓** | **Pattern** | ***sup*↓** | ***uo*** |
| $(abcd)$ | 2 | 0.9081 | $(abcd)$ | 2 | 0.9081 | $(c)$ | 8 | 0.6468 |
| $(acd)$ | 4 | 0.8972 | $(acd)$ | 4 | 0.8972 | $(d)$ | 7 | 0.0897 |
| $(ce)$ | 3 | 0.8776 | $(ce)$ | 3 | 0.8776 | $(a)$ | 5 | 0.2985 |
| $(abc)$ | 2 | 0.8308 | $(abc)$ | 2 | 0.8308 | $(b)$ | 5 | 0.2192 |
| $(ac)$ | 4 | 0.8273 | $(ac)$ | 4 | 0.8273 | $(ad)$ | 5 | 0.3609 |
| $(abde)$ | 2 | 0.7755 | $(ade)$ | 2 | 0.6979 | $(cd)$ | 5 | 0.6881 |
| $(abe)$ | 2 | 0.7081 | $(bc)$ | 3 | 0.6554 | $(e)$ | 4 | 0.4022 |
| $(ade)$ | 2 | 0.6979 | $(c)$ | 8 | 0.6468 | $(ac)$ | 4 | 0.8273 |
| $(cd)$ | 5 | 0.6881 | $(ae)$ | 2 | 0.6305 | $(bd)$ | 4 | 0.3620 |
| $(bc)$ | 3 | 0.6554 | $(bcd)$ | 2 | 0.5604 | $(acd)$ | 4 | 0.8972 |

We further compare the number of patterns on the test datasets to show the effect of varying the parameters $\alpha$ and $\beta$. Results for various parameter values are shown in TABLE IV and TABLE V, respectively. Notice that the parameters in TABLE IV are varying $\alpha$ under a fixed $\beta$ = 0.3, and the parameter settings on each dataset are the same as them in Fig. 4. In TABLE V, the parameters are varying $\beta$ under a fixed $\alpha$, and details of the parameter settings in each dataset are the same as them in Fig. 5.

From TABLE IV and TABLE V, it can be clearly observed that sets of derived patterns are quite different for various $\alpha$ or $\beta$ values. The number of HUOPs* is always smaller than that of HUOPs. For example, on chess dataset as shown in TABLE IV and TABLE V, most of final HUOPs are missed by the OCEAN algorithm since the number of HUOPs* is always considerably smaller than that of HUOPs. It means that numerous interesting high utility occupancy patterns are

TABLE IV
DERIVED PATTERNS UNDER VARIED $\alpha$

| **Dataset** | **Patterns** | **# patterns by varying threshold $\alpha$** | | | | | |
|---|---|---|---|---|---|---|---|
| | | $\alpha_1$ | $\alpha_2$ | $\alpha_3$ | $\alpha_4$ | $\alpha_5$ | $\alpha_6$ |
| BMSPOS2 | HUOPs* | 29,389 | 17,005 | 11,876 | 8,961 | 7,065 | 5,884 |
| ($\beta$: 0.3) | HUOPs | 29,444 | 17,048 | 11,899 | 8,987 | 7,094 | 5,905 |
| retail | HUOPs* | 12,801 | 8,155 | 5,902 | 4,513 | 3,989 | 3,282 |
| ($\beta$: 0.3) | HUOPs | 13,105 | 8,209 | 5,927 | 4,527 | 4,004 | 3,291 |
| chess | HUOPs* | 2,423 | 1,776 | 1,374 | 1,108 | 903 | 695 |
| ($\beta$: 0.3) | HUOPs | 11,469 | 8,949 | 7,075 | 5,510 | 4,273 | 3,351 |
| mushroom | HUOPs* | 17,678 | 12,429 | 10,739 | 10,252 | 2,821 | 1,199 |
| ($\beta$: 0.3) | HUOPs | 52,795 | 33,629 | 31,741 | 31,025 | 4,613 | 1,672 |
| T10I4D100K | HUOPs* | 536,414 | 246,932 | 138,877 | 91,557 | 67,834 | 53,957 |
| ($\beta$: 0.3) | HUOPs | 550,676 | 253,617 | 144,106 | 96,505 | 72,555 | 59,068 |
| T40I10D100K | HUOPs* | 33,171 | 14,415 | 10,465 | 258 | 151 | 55 |
| ($\beta$: 0.3) | HUOPs | 71,941 | 15,496 | 12,273 | 1,367 | 1,002 | 548 |

TABLE V
DERIVED PATTERNS UNDER VARIED $\beta$

| **Dataset** | **Patterns** | **# patterns by varying threshold $\beta$** | | | | | |
|---|---|---|---|---|---|---|---|
| | | $\beta_1$ | $\beta_2$ | $\beta_3$ | $\beta_4$ | $\beta_5$ | $\beta_6$ |
| BMSPOS2 | HUOPs* | 287,850 | 106,239 | 34,195 | 11,876 | 5,089 | 2,696 |
| ($\alpha$: 0.02%) | HUOPs | 287,902 | 106,285 | 34,233 | 11,899 | 5,115 | 2,721 |
| retail | HUOPs* | 31,036 | 13,910 | 5,902 | 2,295 | 842 | 315 |
| ($\alpha$: 0.014%) | HUOPs | 31,067 | 13,940 | 5,927 | 2,335 | 883 | 346 |
| chess | HUOPs* | 9,820 | 4,837 | 1,374 | 514 | 125 | 14 |
| ($\alpha$: 62%) | HUOPs | 24,905 | 14,050 | 7,075 | 3,153 | 1,170 | 357 |
| mushroom | HUOPs* | 16,733 | 10,739 | 4,914 | 1,327 | 505 | 136 |
| ($\alpha$: 18%) | HUOPs | 41,311 | 31,741 | 21,755 | 12,917 | 6,431 | 2,556 |
| T10I4D100K | HUOPs* | 181,541 | 131,631 | 91,557 | 61,079 | 38,980 | 23,435 |
| ($\alpha$: 0.012%) | HUOPs | 182,538 | 135,710 | 96,505 | 66,772 | 44,380 | 27,906 |
| T40I10D100K | HUOPs* | 60,749 | 37,163 | 21,105 | 10,465 | 3,898 | 911 |
| ($\alpha$: 0.8%) | HUOPs | 68,457 | 42,887 | 24,592 | 12,273 | 5,031 | 1,594 |

effectively discovered by HUOPM algorithm, while most of them are missed by the OCEAN algorithm. In other words, although the addressed HUOPM problem can capture high qualified patterns well, OCEAN fails to discover the complete set of HUOPs. It can also be observed that the number of produced patterns (both HUOPs* and HUOPs), decrease when the minimum support threshold is increased. And less HUOPs* and HUOPs are obtained when $\beta$ is set higher. Besides, the number of missing patterns which is caused by OCEAN (i.e., HUOPs - HUOPs*) sometimes increases when varying $\beta$, while it sometimes decreases, as shown on the chess and T10I4D100K datasets when varying $\beta$ with a fixed $\alpha$. From the above analysis of the results of found patterns, it can be concluded that the proposed HUOPM algorithm for mining HUOPs is acceptable and can solve a serious shortcoming of the state-of-the-art OCEAN algorithm.

### C. Efficiency Analysis

To evaluate the efficiency of OCEAN and the proposed HUOPM algorithm on various datasets, a performance comparison of the different strategies used in HUOPM is presented next in terms of execution time. The results in terms of runtime for various parameters are shown in Fig. 4 and Fig. 5, respectively.

From Fig. 4 and Fig. 5, it can be clearly observed that the runtime of OCEAN is the worst, compared to the other algorithms in most cases, and that the HUOPM$_{P1234}$ algorithm

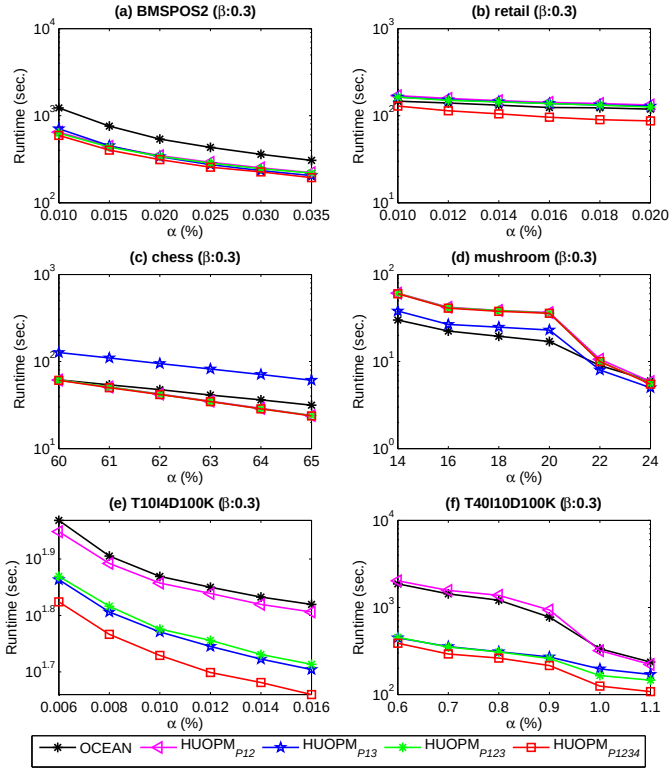



Fig. 4. Runtime under varied $\alpha$ with a fixed $\beta$.

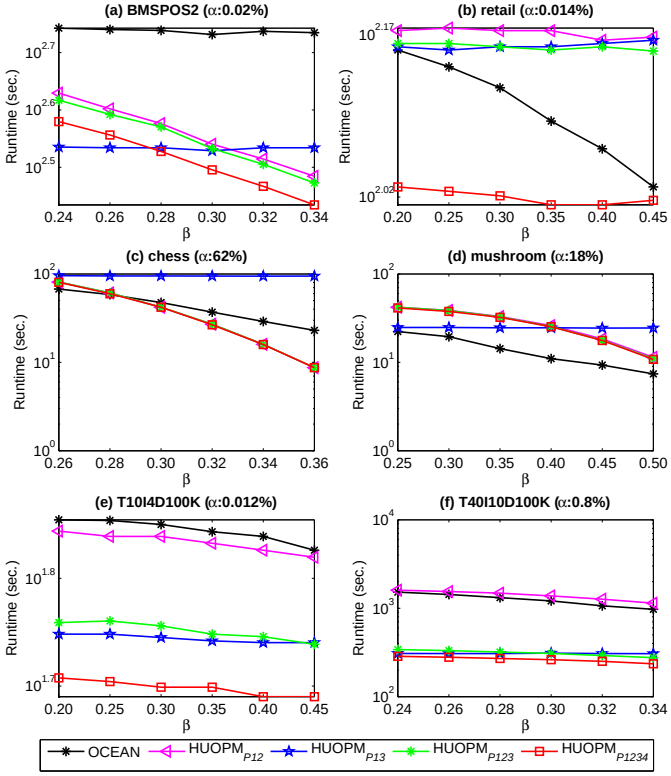



Fig. 5. Runtime under varied $\beta$ with a fixed $\alpha$.

(which adopts all pruning strategies) is the fastest on all datasets in all cases. We also draw the following conclusions. (1) The difference between HUOPM$_{P12}$ and HUOPM$_{P123}$ indicates that the Strategy 3 always reduces the search space by pruning subtrees, as it can be observed in Fig. 4 and Fig. 5. (2) By comparing HUOPM$_{P13}$ and HUOPM$_{P123}$, it can be concluded that the Strategy 2, using the upper bound of utility occupancy, provides a trade-off between efficiency and effectiveness. For example, consider results for BMSPOS2, shown in Fig. 5(a). When $\beta$ = 0.24, HUOPM$_{P13}$ is faster than HUOPM$_{P123}$; while HUOPM$_{P13}$ consumes more runtime than HUOPM$_{P123}$ when $\beta$ = 0.32 and $\beta$ = 0.34. The reason for this behavior is that it needs to spend additional time to calculate the upper bounds, but sometimes unpromising itemsets can be directly pruned by other pruning strategies. (3) In general, the Strategy 4 can significantly reduce the execution time. For example, as shown in Fig. 4(b), (d) and (f), Fig. 5(a), (b) and (e), the runtime of HUOPM$_{P1234}$ is considerably smaller than that of HUOPM$_{P123}$. Therefore, the HUOPM algorithm applies four pruning strategies to prune unpromising patterns early, which greatly speed up the mining efficiency, compared to the baseline algorithms and the OCEAN algorithm.

Furthermore, we evaluate the statistical analysis [51] by two-way ANOVA test [52] to show whether the proposed HUOPM has a significant difference to the traditional OCEAN algorithm. Based on a two-way ANOVA analysis, in most cases, there is a significant difference of the runtime between HUOPM and OCEAN under varied $\alpha$ with a fixed $\beta$. Details are described below: $F$ = 11.551, $P$ = 0.019 $<$ 0.050, in Fig. 4(a); $F$ = 156.372, $P$ $<$ 0.001, in Fig. 4(b); $F$ = 3.619, $P$ = 0.116, in Fig. 4(c); $F$ = 8.871, $P$ = 0.031 $<$ 0.050, in Fig. 4(d); $F$ = 491.368, $P$ $<$ 0.001, in Fig. 4(e); $F$ = 11.597, $P$ = 0.019 $<$ 0.050, in Fig. 4(f). Consider Fig. 5, the results of a two-way ANOVA analysis are presented below. $F$ = 211.304, $P$ $<$ 0.001, in Fig. 5(a); $F$ = 18.649, $P$ = 0.008 $<$ 0.010, in Fig. 5(b); $F$ = 1.303, $P$ = 0.305, in Fig. 5(c); $F$ = 27.371, $P$ = 0.003 $<$ 0.050, in Fig. 5(d); $F$ = 11705.343, $P$ $<$ 0.001, in Fig. 5(e); $F$ = 157.518, $P$ $<$ 0.001, in Fig. 5(f). Thus, there is a significant difference of the runtime between the proposed HUOPM and the OCEAN algorithm under different parameter settings.

### D. Effect of Pruning Strategies

To assess the impact of pruning strategies, we further compared the number of nodes visited in the FU-tree. The number of nodes visited by the four versions of the proposed algorithm (HUOPM$_{P12}$, HUOPM$_{P13}$, HUOPM$_{P123}$ and HUOPM$_{P1234}$) are denoted as $N_1$, $N_2$, $N_3$ and $N_4$, respectively. Results for the same parameter settings as in previous experiments are shown in Fig. 6 and Fig. 7, respectively.

Based on Fig. 4 to Fig. 7, five observations are made. (1) HUOPM prunes a larger part of the search space in terms of number of nodes and is faster when $\alpha$ and $\beta$ are increased. In general, the runtime and the number of visited nodes in the search tree of HUOPM decrease as $\alpha$ or $\beta$ increases. Therefore, the effect of the pruning strategies on

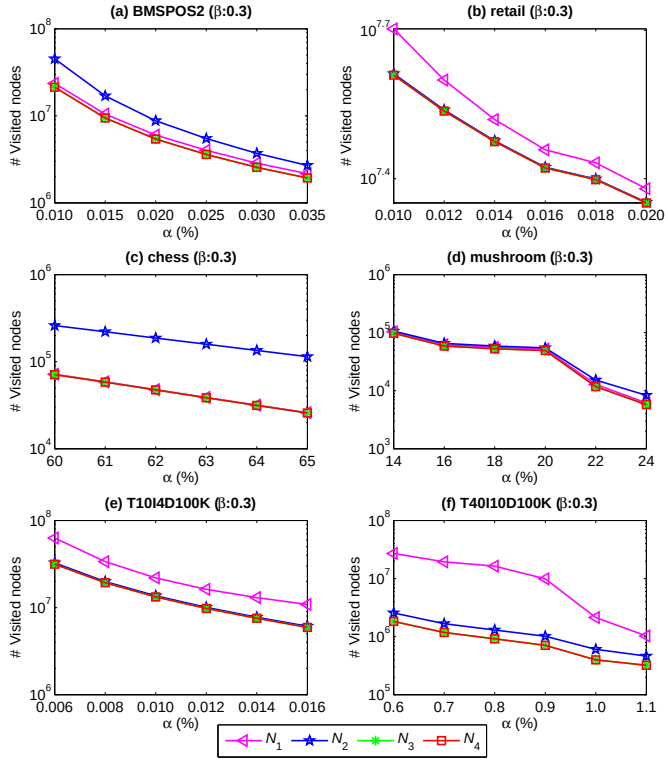



Fig. 6. The number of visited nodes under varied $\alpha$ with a fixed $\beta$.

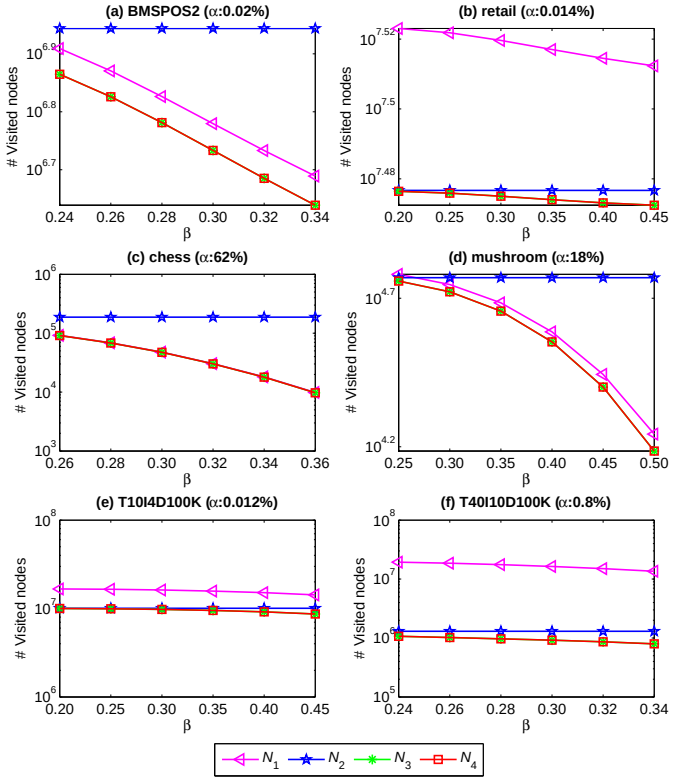



Fig. 7. The number of visited nodes under varied $\beta$ with a fixed $\alpha$.

the number of visited nodes, to some extent, reflects the execution time required by the algorithms. Hence, a smaller tree is more quickly processed since less execution time is required for spanning the FU-tree and constructing UO-lists. (2) Comparing $N_2$ and $N_3$ (with and without the upper bound) indicates that the proposed upper bound is useful for pruning unpromising patterns, thus leading to a more compact search space on both sparse and dense datasets. (3) The numbers $N_3$ and $N_4$ are always the same regardless of the increase in $\alpha$ and $\beta$. It indicates that the Strategy 4 does not filter more unpromising patterns when compared to Strategy 3, but it has a powerful effect on reducing the runtime. We can see this trend more clearly in Fig. 4 and Fig. 5 by observing the difference between HUOPM$_{P123}$ and HUOPM$_{P1234}$. Thus, HUOPM$_{P1234}$ always consumes the least runtime among the four versions of the algorithm. (4) The higher $\alpha$ and $\beta$ are set, the smaller the search space (number of visited nodes) is. Thus, the less runtime is required. Moreover, the less memory usage is required, but we omit the detailed memory consumption results due to the page limit. (5) The utility occupancy is a significant factor for improving mining performance in the addressed mining task.

### *E. Processing Order of Items*

Since the processing order of items may influence the performance of a mining algorithm, it is important to select a suitable sorting order for the proposed algorithm. To assess how different processing orders influence the performance of the proposed HUOPM algorithm, we measured the runtime and memory consumption of HUOPM with all pruning strategies but with different processing orders. Five types of processing orders are evaluated: the lexicographic order (denoted as HUOPM$_{lexi}$), the transaction-weighted utilization ascending order (denoted as HUOPM$_{twuas}$), the transaction-weighted utilization descending order (denoted as HUOPM$_{twude}$), the support ascending order (denoted as HUOPM$_{supas}$), and the support ascending descending order (denoted as HUOPM$_{supde}$).

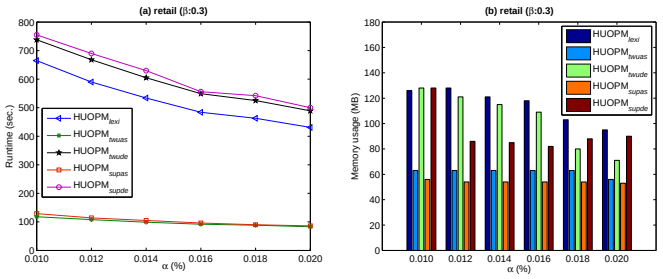



Fig. 8. Effect of processing order of items (Five types of processing orders: HUOPM$_{lexi}$, HUOPM$_{twuas}$, HUOPM$_{twude}$, HUOPM$_{supas}$ and HUOPM$_{supde}$).

Fig. 8 shows the experimental results on the accidents and retail datasets, respectively. As we can see, HUOPM$_{twuas}$ has similar performance when compared to HUOPM$_{supas}$. However, HUOPM$_{twuas}$ requires more memory usage than HUOPM$_{supas}$, as shown in Fig. 8(b). Thus, the adopted

support ascending order (HUOPM$_{supas}$) always leads to the best performance in terms of execution time and memory usage. Clearly, the *TWU* or support descending order provides the worst performance. It indicates that the adopted processing order w.r.t. support ascending order of items in the proposed HUOPM approach can greatly reduce the number of UO-lists and join operations performed for a mining task.

## VI. Conclusions and Future Work

In this paper, we propose an effective and efficient HUOPM algorithm to address a new research problem of mining high utility occupancy patterns with utility occupancy. The utility occupancy can lead to useful patterns that contribute a large portion of total utility for each individual transaction representing user interests or user habit. To the best of our knowledge, no prior algorithms address this problem successfully and effectively. It may provide a new research perspective for utility mining, to a certain extent. In HUOPM, an upper bound of utility occupancy for all possible extensions rooted at a processed node/itemset can be quickly derived by utilizing the UO-list. Based on the developed two *downward closure* properties and four pruning strategies, the HUOPM algorithm can directly discover the profitable HUOPs from the FU-tree using UO-list without candidate generation. Extensive experiments on several datasets show that HUOPM can efficiently find the complete set of high utility occupancy patterns and significantly outperforms the state-of-the-art OCEAN algorithm. Moreover, the proposed pruning strategies are powerful to prune the search space and speed up the mining performance. Extending HUOPM to address the distributed mining problem [5], dynamic mining [40], and privacy preserving issue [53] are part of our future work.

## References


[1] R. Agrawal, R. Srikant *et al.*, "Fast algorithms for mining association rules," in *Proceedings of the 20th International Conference on Very Large Data Bases*, vol. 1215, 1994, pp. 487–499.

[2] J. Han, J. Pei, Y. Yin, and R. Mao, "Mining frequent patterns without candidate generation: a frequent-pattern tree approach," *Data Mining and Knowledge Discovery*, vol. 8, no. 1, pp. 53–87, 2004.

[3] L. Geng and H. J. Hamilton, "Interestingness measures for data mining: A survey," *ACM Computing Surveys*, vol. 38, no. 3, p. 9, 2006.

[4] M. S. Chen, J. Han, and P. S. Yu, "Data mining: an overview from a database perspective," *IEEE Transactions on Knowledge and data Engineering*, vol. 8, no. 6, pp. 866–883, 1996.

[5] W. Gan, J. C. W. Lin, H. C. Chao, and J. Zhan, "Data mining in distributed environment: a survey," *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, vol. 7, no. 6, p. e1216, 2017.

[6] K. Gouda and M. J. Zaki, "GenMax: an efficient algorithm for mining maximal frequent itemsets," *Data Mining and Knowledge Discovery*, vol. 11, no. 3, pp. 223–242, 2005.

[7] E. R. Omiecinski, "Alternative interest measures for mining associations in databases," *IEEE Transactions on Knowledge and Data Engineering*, vol. 15, no. 1, pp. 57–69, 2003.

[8] J. Pei, J. Han, and L. V. Lakshmanan, "Mining frequent itemsets with convertible constraints," in *Proceedings of 17th International Conference on Data Engineering*. IEEE, 2001, pp. 433–442.

[9] J. C. W. Lin, W. Gan, P. Fournier-Viger, T. P. Hong, and V. S. Tseng, "Weighted frequent itemset mining over uncertain databases," *Applied Intelligence*, vol. 44, no. 1, pp. 232–250, 2016.

[10] W. Gan, J. C. W. Lin, P. Fournier-Viger, H. C. Chao, V. S. Tseng, and P. S. Yu, "A survey of utility-oriented pattern mining," *arXiv preprint arXiv:1805.10511*, 2018.

[11] C. F. Ahmed, S. K. Tanbeer, B. S. Jeong, and Y. K. Lee, "Efficient tree structures for high utility pattern mining in incremental databases," *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, no. 12, pp. 1708–1721, 2009.

[12] M. Liu and J. Qu, "Mining high utility itemsets without candidate generation," in *Proceedings of the 21st ACM International Conference on Information and Knowledge Management*. ACM, 2012, pp. 55–64.

[13] V. S. Tseng, C. W. Wu, B. E. Shie, and P. S. Yu, "UP-Growth: an efficient algorithm for high utility itemset mining," in *Proceedings of the 16th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2010, pp. 253–262.

[14] V. S. Tseng, B. E. Shie, C. W. Wu, and P. S. Yu, "Efficient algorithms for mining high utility itemsets from transactional databases," *IEEE Transactions on Knowledge and Data Engineering*, vol. 25, no. 8, pp. 1772–1786, 2013.

[15] R. Chan, Q. Yang, and Y. D. Shen, "Mining high utility itemsets," in *Proceedings of the 3rd IEEE International Conference on Data Mining*. IEEE, 2003, pp. 19–26.

[16] H. Yao, H. J. Hamilton, and C. J. Butz, "A foundational approach to mining itemset utilities from databases," in *Proceedings of the SIAM International Conference on Data Mining*. SIAM, 2004, pp. 482–486.

[17] L. Tang, L. Zhang, P. Luo, and M. Wang, "Incorporating occupancy into frequent pattern mining for high quality pattern recommendation," in *Proceedings of the 21st ACM International Conference on Information and Knowledge Management*. ACM, 2012, pp. 75–84.

[18] B. Shen, Z. Wen, Y. Zhao, D. Zhou, and W. Zheng, "OCEAN: fast discovery of high utility occupancy itemsets," in *Pacific-Asia Conference on Knowledge Discovery and Data Mining*. Springer, 2016, pp. 354–365.

[19] M. Zihayat, H. Davoudi, and A. An, "Mining significant high utility gene regulation sequential patterns," *BMC Systems Biology*, vol. 11, no. 6, p. 109, 2017.

[20] J. M. Luna, A. Cano, M. Pechenizkiy, and S. Ventura, "Speeding-up association rule mining with inverted index compression," *IEEE Transactions on Cybernetics*, vol. 46, no. 12, pp. 3059–3072, 2016.

[21] R. Srikant and R. Agrawal, "Mining quantitative association rules in large relational tables," in *ACM SIGMOD Record*, vol. 25, no. 2. ACM, 1996, pp. 1–12.

[22] T. P. Hong, C. S. Kuo, and S. C. Chi, "Mining association rules from quantitative data," *Intelligent Data Analysis*, vol. 3, no. 5, pp. 363–376, 1999.

[23] H. Verlinde, M. De Cock, and R. Boute, "Fuzzy versus quantitative association rules: a fair data-driven comparison," *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, vol. 36, no. 3, pp. 679–684, 2005.

[24] H. Yao and H. J. Hamilton, "Mining itemset utilities from transaction databases," *Data & Knowledge Engineering*, vol. 59, no. 3, pp. 603–626, 2006.

[25] F. Tao, F. Murtagh, and M. Farid, "Weighted association rule mining using weighted support and significance framework," in *Proceedings of the 9th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2003, pp. 661–666.

[26] J. C. W. Lin, W. Gan, P. Fournier-Viger, and T. P. Hong, "RWFIM: recent weighted-frequent itemsets mining," *Engineering Applications of Artificial Intelligence*, vol. 45, pp. 18–32, 2015.

[27] Y. Liu, W. K. Liao, and A. Choudhary, "A two-phase algorithm for fast discovery of high utility itemsets," in *Pacific-Asia Conference on Knowledge Discovery and Data Mining*. Springer, 2005, pp. 689–695.

[28] J. Liu, K. Wang, and B. C. Fung, "Direct discovery of high utility itemsets without candidate generation," in *Proceedings of the IEEE 12th International Conference on Data Mining*. IEEE, 2012, pp. 984–989.

[29] P. Fournier-Viger, C. W. Wu, S. Zida, and V. S. Tseng, "FHM: faster high-utility itemset mining using estimated utility co-occurrence pruning," in *International Symposium on Methodologies for Intelligent Systems*. Springer, 2014, pp. 83–92.

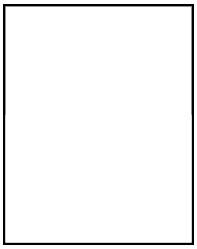
[30] S. Krishnamoorthy, "Pruning strategies for mining high utility itemsets," *Expert Systems with Applications*, vol. 42, no. 5, pp. 2371–2381, 2015.

[31] S. Zida, P. Fournier-Viger, J. C. W. Lin, C. W. Wu, and V. S. Tseng, "EFIM: a fast and memory efficient algorithm for high-utility itemset mining," *Knowledge and Information Systems*, vol. 51, no. 2, pp. 595–625, 2017.

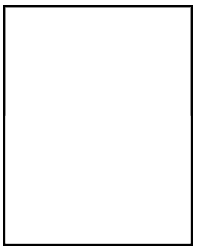
[32] J. C. W. Lin, W. Gan, P. Fournier-Viger, T. P. Hong, and V. S. Tseng, "Efficient algorithms for mining high-utility itemsets in uncertain databases," *Knowledge-Based Systems*, vol. 96, pp. 171–187, 2016.

[33] ——, "Efficiently mining uncertain high-utility itemsets," *Soft Computing*, vol. 21, no. 11, pp. 2801–2820, 2017.

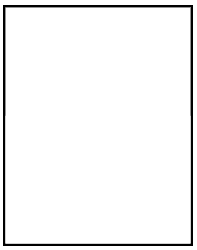
[34] J. C. W. Lin, W. Gan, T. P. Hong, and V. S. Tseng, "Efficient algorithms for mining up-to-date high-utility patterns," *Advanced Engineering Informatics*, vol. 29, no. 3, pp. 648–661, 2015.
[35] J. C. W. Lin, J. Zhang, P. Fournier-Viger, T. P. Hong, and J. Zhang, "A two-phase approach to mine short-period high-utility itemsets in transactional databases," *Advanced Engineering Informatics*, vol. 33, pp. 29–43, 2017.
[36] J. C. W. Lin, P. Fournier-Viger, and W. Gan, "FHN: an efficient algorithm for mining high-utility itemsets with negative unit profits," *Knowledge-Based Systems*, vol. 111, pp. 283–298, 2016.
[37] J. C. W. Lin, W. Gan, T. P. Hong, and B. Zhang, "An incremental high-utility mining algorithm with transaction insertion," *The Scientific World Journal*, vol. 2015, 2015.
[38] J. C. W. Lin, W. Gan, and T. P. Hong, "A fast maintenance algorithm of the discovered high-utility itemsets with transaction deletion," *Intelligent Data Analysis*, vol. 20, no. 4, pp. 891–913, 2016.
[39] ——, "A fast updated algorithm to maintain the discovered high-utility itemsets for transaction modification," *Advanced Engineering Informatics*, vol. 29, no. 3, pp. 562–574, 2015.
[40] W. Gan, J. C. W. Lin, P. Fournier-Viger, H. C. Chao, T. P. Hong, and H. Fujita, "A survey of incremental high-utility itemset mining," *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, vol. 8, no. 2, p. e1242, 2018.
[41] U. Yun, G. Lee, and E. Yoon, "Efficient high utility pattern mining for establishing manufacturing plans with sliding window control," *IEEE Transactions on Industrial Electronics*, vol. 64, no. 9, pp. 7239–7249, 2017.
[42] J. C. W. Lin, W. Gan, P. Fournier-Viger, T. P. Hong, and V. S. Tseng, "Fast algorithms for mining high-utility itemsets with various discount strategies," *Advanced Engineering Informatics*, vol. 30, no. 2, pp. 109–126, 2016.
[43] W. Gan, J. C. W. Lin, P. Fournier-Viger, H. C. Chao, and H. Fujita, "Extracting non-redundant correlated purchase behaviors by utility measure," *Knowledge-Based Systems*, vol. 143, pp. 30–41, 2018.
[44] V. S. Tseng, C. W. Wu, P. Fournier-Viger, and P. S. Yu, "Efficient algorithms for mining top-$k$ high utility itemsets," *IEEE Transactions on Knowledge and Data Engineering*, vol. 28, no. 1, pp. 54–67, 2016.
[45] J. C. W. Lin, W. Gan, P. Fournier-Viger, T. P. Hong, and H. C. Chao, "FDHUP: fast algorithm for mining discriminative high utility patterns," *Knowledge and Information Systems*, vol. 51, no. 3, pp. 873–909, 2017.
[46] Y. C. Lin, C. W. Wu, and V. S. Tseng, "Mining high utility itemsets in big data," in *Pacific-Asia Conference on Knowledge Discovery and Data Mining*. Springer, 2015, pp. 649–661.
[47] N. Pasquier, Y. Bastide, R. Taouil, and L. Lakhal, "Pruning closed itemset lattices for association rules," in *Proceedings of International Conference on Advanced Databases*, 1998, pp. 177–196.
[48] R. Rymon, "Search through systematic set enumeration," 1992.
[49] "Frequent itemset mining dataset repository. http://fimi.ua.ac.be/data/," 2012.
[50] "Quest synthetic data generator. http://www.Almaden.ibm.com/cs/quest/syndata.html," 1994.
[51] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *Journal of Machine Learning Research*, vol. 7, pp. 1–30, 2006.
[52] T. H. Wonnacott and R. J. Wonnacott, *"Introductory statistics"*. Wiley New York, 1990.
[53] W. Gan, J. C. W. Lin, H. C. Chao, S. L. Wang, and P. S. Yu, "Privacy preserving utility mining: a survey," *arXiv preprint arXiv:1811.07389*, 2018.



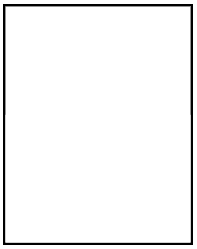
**Wensheng Gan** is currently a joint Ph.D. candidate at the Department of Computer Science, University of Illinois at Chicago, IL, USA; and a Ph.D. candidate at the School of Computer Science and Technology, Harbin Institute of Technology (Shenzhen), Guangdong, China. He received B.S. and M.S. degrees in Computer Science from South China Normal University and Harbin Institute of Technology (Shenzhen), Guangdong, China, in 2013 and 2015, respectively. His research interests include data mining, utility mining and Big Data technologies. He has published more than 50 research papers in peer-reviewed journals and international conferences.



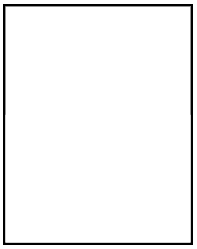
**Jerry Chun-Wei Lin** is an associate professor at Department of Computing, Mathematics and Physics, Western Norway University of Applied Sciences, Bergen, Noway. He received the Ph.D. in Computer Science and Information Engineering, National Cheng Kung University, Tainan, Taiwan in 2010. His research interests include data mining, privacy-preserving and security, big data analytics, and machine learning. He has published more than 250 research papers in peer-reviewed international conferences and journals. He is the co-leader of the popular SPMF open-source data mining library, the funder of the PPSF open-source privacy-preserving and security project, and Editor-in-Chief of the *Data Mining and Pattern Recognition* (DSPR) journal.



**Philippe Fournier-Viger** is a full professor and Youth 1000 scholar at the Harbin Institute of Technology (Shenzhen), Shenzhen, China. He received the Ph.D. in Computer Science from the University of Quebec, Montreal, in 2010. His research interests include data mining, pattern mining, sequence analysis and prediction, e-learning, and social network mining. He has published more than 200 research papers in peer-reviewed international conferences and journals. He is the founder of the popular SPMF open-source data-mining library, which has been cited in more than 700 research papers since 2010. He is also Editor-in-Chief of the *Data Mining and Pattern Recognition* (DSPR) journal.



**Han-Chieh Chao** has been the president of National Dong Hwa University since February 2016. He received M.S. and Ph.D. degrees in Electrical Engineering from Purdue University in 1989 and 1993, respectively. His research interests include high-speed networks, wireless networks, IPv6-based networks, and artificial intelligence. He has published nearly 500 peer-reviewed professional research papers. He is the Editor-in-Chief (EiC) of IET Networks' *Journal of Internet Technology*. He is the founding EiC of the *International Journal of Internet Protocol Technology* and the *International Journal of Ad Hoc and Ubiquitous Computing*. Dr. Chao has served as the guest editor for ACM MONET, IEEE JSAC, *IEEE Communications Magazine*, *IEEE Systems Journal*, *Computer Communications*, *IEEE Proceedings Communications*, *Wireless Personal Communications*, and *Wireless Communications & Mobile Computing*. Dr. Chao is an IEEE Senior Member and a fellow of IET.



**Philip S. Yu** received the B.S. degree in Electrical Engineering from National Taiwan University, M.S. and Ph.D. degrees in Electrical Engineering from Stanford University, and an MBA from New York University. He is a distinguished professor of computer science with the University of Illinois at Chicago (UIC) and also holds the Wexler Chair in Information Technology at UIC. Before joining UIC, he was with IBM, where he was manager of the Software Tools and Techniques Department at the Thomas J. Watson Research Center. His research interests include data mining, data streams, databases, and privacy. He has published more than 1,200 papers in peer-reviewed journals (i.e., TKDE, TKDD, VLDBJ, ACM TIST) and conferences (KDD, ICDE, WWW, AAAI, SIGIR, ICML, CIKM, etc). He holds or has applied for more than 300 U.S. patents. Dr. Yu is the Editor-in-Chief of *ACM Transactions on Knowledge Discovery from Data*. He received the ACM SIGKDD 2016 Innovation Award for his influential research and scientific contributions on mining, fusion, and anonymization of Big Data, and the IEEE Computer Society 2013 Technical Achievement Award. Dr. Yu is a fellow of the ACM and the IEEE.